

AI 应用能力文档

出海达人营销投前决策智能体

纪辉（Hugh） | 天津大学 计算机科学与技术 2023 级 | 应聘岗位：AI Builder（产品向）

把一句话投放 brief，变成一份可执行、可追责、带预算分配和风险证据链的达人投放清单，并用可复现的实验说清这份清单比「凭粉丝量选人」少浪费多少钱——以及这个结论有多稳。

这份文档里的每一个数字都有出处：绝大部分能在仓库里用一条命令复跑（固定种子、无网络依赖、两次运行产物逐字节相同）；少数标明「服务实测」的来自 HTTP 服务，在仓库里 `cd api && python3 main.py` 起服务后可自己 `curl` 复核。凡是我无法复现的数字，我不会写进来。

Demo：`cd web && pnpm install && pnpm run prepare-data && pnpm run dev`（仓库已配好 GitHub Actions，可一键发布到 GitHub Pages）

HTTP 服务：`pip install -r api/requirements.txt` 后 `cd api && python3 main.py` 本地起（`api/run.sh` 是部署平台生成的脚本，本地不用它），无需鉴权即可 `curl`；部署包见 `deploy/hf-space/`
代码仓库与完整技术文档（docs/01~06，约 3.4 万字）随本文档一并提交

提交入口只有一个 [GitHub 仓库](#)：Demo（一条命令本地起，或走仓库已配好的 GitHub Pages workflow）、HTTP 服务、本文档 PDF、全部源码与实验产物、六篇技术分册（docs/01~docs/06，约 3.4 万字）都在仓库里，仓库 README 顶部有按阅读时长分流的导航。

0. 如果只看一页

做了什么：把一句话投放 brief（"3C 小家电新品要投北美，8 万美元，看转化"），变成一份可执行、可追责、带预算分配和风险证据链的达人投放清单，并用可复现的实验说清这份清单比行业最朴素的做法——[按粉丝量降序选人直到花完预算](#)——少浪费多少钱。

为什么是这个题：出海营销的钱，很大一部分不是浪费在投后优化不够，而是浪费在[投前选错人](#)。达人自称做 3C、实际内容在做美妆；粉丝 200 万但互动率 0.3%；受众 80% 在印尼而你要投沙特。这些问题投后看数据能发现，但钱已经花完了。

最硬的三个事实：

线上跑的是真链路，而且它被另一套实现盯着	FastAPI 服务无鉴权可调（本地 <code>cd api && python3 main.py</code> 即起）， 门禁/预算/审计/证据句 全部 <code>import src/koxpilot</code> ， 不是为接口重写一遍 。同一批 <code>kox_id</code> 由浏览器内 TS 引擎独立重算并 逐条对撞 ：4 条 brief 合计 1,431 条判定 、 差异 0 条 ；构建期另有全库 5,000 条、 差异 0 条 的离线比对
价值结论由标注结果裁判，不是自己给自己打分	浪费金额与有效曝光 一律按 ground truth 结算 （ <code>method.judge</code> 写死"防自证"）。\$245,000 预算下有效曝光 +98.64% ；把水号曝光从"按 0 计"放宽到"按 50% 计"，仍是 +64.71% ——两个口径同向，结论不吃那条假设
该不该用 LLM 是算过的，不是表态	G0/G1 纯规则、仅 brief 解析与语义适配用 LLM，调用量从全 LLM 的 888 次降到 225 次 (1/3.95) ；同时用控制变量实验证明 判据写不清时 LLM 不如规则 （v1/v2 都输给零成本 Jaccard 规则）

门禁能力的跨种子稳定性（F1 [0.7275 ± 0.0090](#)、AUC [0.8878 ± 0.0054](#)，12 种子 CV ≤ 0.023）与价值幅度的显著性检验（[21.5% ± 13.7%](#)，并承认 1/12 个种子跑输基线）见 § 3、§ 4；假设敏感时的[分层交付降级](#)路径见 § 6。

最诚实的弱项（我把它们放在第一页，不是附录）：

- [数据是我自己合成的](#)，不是真实达人数据。所有绝对数值只在这套数据生成假设内成立。这是这个作品最大的边界。
- [阈值敏感性判定为"不稳健"](#)：阈值 ±20% 时最大 F1 偏移 0.0582，我如实标了 `stable=False` 而不是调宽判据让它变 `True`。
- 我原本关于自己产品价值来源的判断，[被自己的实验推翻了](#)（详见 § 4.3）。
- [我自己加的对照臂在主指标上打赢了我](#)：`diversified_no_gate`（只做结构分散、不做门禁）的每美元有效曝光在三条 brief 上分别比我高 [47.4% / 47.5% / 44.0%](#)。代价是它 18.25% 的预算打水漂、41 个水号、4 个品牌安全高风险号、228 份合约。[门禁买的是确定性和品牌安全，不是每美元曝光效率的上限](#)——这条连同它对我更不利的细节（BRIEF-002 上我的浪费率比它还高）写在 § 4.4，我没有把它藏进附录。

5. 服务是演示实例，不是生产部署：单实例、无副本、无灰度，冷启动的第一个请求要等数据集懒加载（之后常温 /api/health 约 120ms）。它只读数据、不写任何东西。
6. A1 这次实测走的是规则解析，不是 LLM——但原因不是“没配凭据”：/api/health 返回的 llm_runtime.available 其实是 true（环境里确实有 openai 凭据），真调用被网关拒了：HTTP 403 model is not allowed。于是 /api/plan 的 brief.parse_path 落到 rule，而 brief.notes 里逐字写着“模型调用失败（openai：… permission_denied … HTTP 403），已自动降级为确定性规则解析”。准确的表述是：凭据存在但该模型不被允许调用，A1 因此走规则解析，降级原因由服务逐字写进响应而不是吞掉。机制本身是“探到可用凭据就真调、失败即降级并把原因写出来”——评审换一份有权限的凭据就会走 LLM 路径，这不是功能没做。同一条 brief 交给 LLM 解析 vs 交给规则解析的差异也直接摆在页面上：30 个字段里 27 个一致，差的 3 个全在竞品回避——LLM 会按品类常识补出原文没写的竞品，规则版补不出来。
7. A4 语义适配的正式链路是规则版：LLM 版在构建期真调过并留有 token 账，但覆盖率只有 14.9%，按判据不予升格（§ 8）。

1. 为什么做这个题：与业务的对齐

我看蓝色光标的定位是「AI 驱动的全链路出海营销」，达人/网红营销是其中明确的一环，而 AI Builder（产品向）JD 的第一条职责就落在达人营销场景。所以我没有做一个通用的“AI 助手”，而是选了这条链路上钱最容易白烧、且判断可以被结构化的一个环节：投前选人。

选这个环节还有一个我认为更要紧的理由：投放代理是低毛利生意。在毛利率只有个位数的业务里，“帮你多赚 10%”很难证明，而“帮你少烧 10%”是直接落在利润上的。所以这个作品从一开始就不是往“效果提升”上做叙事，而是往浪费控制与风险兜底上做——这也决定了后面所有指标的选择。

2. 产品是什么

2.1 一条链路，六个 Agent

一句话 brief（自然语言）

- ↓ A1 BriefAgent（LLM 可选）— 自然语言 → 结构化 CampaignSpec；凭据缺失或调用失败即走确定性规则解析
- ↓ A2 RecallAgent — 按品类 / 地域 / 平台 / 粉丝层召回候选
- ↓ A3 GateAgent（规则 + 分位阈值）— 四层质量门禁 G0/G1/G2/G3 → pass/review/reject + 证据链
- ↓ A4 FitAgent — 语义适配打分（人 × 单子）
- ↓ A5 BudgetAgent — 带 5 条硬约束的预算分配
- ↓ A6 AuditAgent — 成本审计 + 反事实价值审计
- ↓

可执行清单：选谁、买几条、花多少、为什么、风险在哪

- ↳ 预算没花完时：结论块说清为什么 + 可执行的放宽建议（§ 2.5）

2.2 线上是怎么跑的：两条通道，互为证据

评审打开的不是一个静态站，也不是录像。浏览器首屏先探活 Python 服务，然后两条通道同时在跑：

[首屏] 浏览器打开页面

└ ① 探活 GET /api/health (800ms 内必须回)

[分支 ready] 通道 A：Python FastAPI 服务

- api/ 直接 import src/koxpilot, A1→A6 全链路真算
- 门禁 gates.engine / 预算 budget.planner / 审计 eval.audit / 证据句 gates.humanize
- 响应带 parity_payload.verdicts (本次召回集每条 kox_id 的判定)

[分支 超时 / 失败 / ok:false] 通道 B 独立接管 (无损降级)

- 浏览器内 TS 引擎跑完 召回 → 门禁 → 预算 全流程，功能不缺项

[服务在线时，两条通道同时在跑]

- ② 浏览器用 TS 引擎对同一批 kox_id 独立重算
- ③ 逐条对撞，界面当场显示比对条数与差异条数

三件事值得单独点出来：

1. 服务不是壳。api/koxpilot_service/ 只做参数校验与形状转换，门禁、预算、审计、证据句全部 import src/koxpilot——一套逻辑，不是两套。/api/kox/{id}/explain 的每一句理由与 CLI koxpilot explain 逐字一致，由 api/check_cli_parity.py 逐字符断言。
2. 对撞是实时的，不是构建期算一次给你看。服务把本次召回集里每个 kox_id 的判定放在 parity_payload.verdicts 回传，浏览器里那套独立写出来的 TS 第二实现重算一遍再逐条比。服务实测：4 条 brief 合计 1,431 条判定、差异 0 条 (web/scripts/verify-service-parity.mjs 打真实服务)。另有构建期全库 5,000 条、差异 0 条的离线比对 (web/public/data/consistency.json)。
3. 界面如实显示当前算的是哪一条通道。三态：Python 服务 / 浏览器引擎 (服务未连接) / 探活中。降级时功能不允许缺失——这是契约里写死的，不是口头承诺。

为什么两条都留着。砍掉 A，就没有"服务端真跑"这回事；砍掉 B，就失去了唯一能证伪 A 的东西。留着两条的代价是我得保证它们不漂移，这正是每次要跑对撞的原因。

2.3 四层门禁：把"这人能不能投"拆成可追责的判据

层	管什么	典型判据
G0	完整性	关键字段缺失/过期，数据不足就不给结论，而不是硬给一个
G1	真实性 (水号)	互动率与同组分位背离、粉丝增长毛刺、评论质量、播放/粉丝比异常
G2	一致性	声称品类 vs 实际内容、受众地域/语言与投放目标错配
G3	品牌安全	敏感品类、争议历史、儿童向标准等

关键设计：阈值是分组标定的，不是全局写死的。instagram 头部号和 youtube 腰部号的"正常互动率"不是一个量级，全局阈值只会同时误杀一批、放过一批。所以阈值按 平台 × 粉丝层 分组、用分位数标定 (样本不足的组显式降级并上报)。

2.4 输出的是"可追责"，不是"一个分数"

每个被拒/被复核的达人都带证据链：命中哪条规则、该组的分位阈值是多少、他的实测值是多少。这一点是我认为产品化的关键——投放执行的人不会因为"模型说 0.31 分"就砍掉一个达人，但会因为"他的互动率在同组里排后 3%，而且评论 60% 是模板句"而砍。

2.5 钱花不出去的时候，它给建议而不是交白卷

这一条是"决策智能体 ≠ 筛选器"最直接的体现。严门禁必然带来一个后果：定向卡得越死，能过审的人越少，预算越可能花不完。一个筛选器到这里就结束——给你 4 个人，剩下 98% 的钱躺着，你自己想办法。

所以当预算利用率低于 60% 时，产品先给一个结论块说清"为什么没花完"，再给最多三条可执行的放宽建议（advice[]）。每条只改一处、其余定向不动，并且是在放宽后的规格上真跑一遍 A2→A3→A5 算出来的，不是估的。

以线上那条自由文本 brief（巴西+墨西哥 Instagram 彩妆，18-24 女性，\$80,000，避开 Focallure）为例，实测召回 182 → 过门禁 7 → 进清单 4 人，只花掉 \$1,249.75。服务给出的结论是："本次通过门禁、且报价与曝光齐备的候选只有 7 人，其中 4 人进了清单；剩余额度已经贴住结构配额，再加钱就会破：单一国家金额占比 ≤ 60%（当前 59.0%）。" 三条建议按增量可花金额降序：

放宽哪一处	增量可花	放宽后利用率	多进来的人
加投相邻市场（BR、MX → 扩到 AR、CO、US）	+\$43,291	55.68%	+24 人
让 review 档带折扣进清单	+\$41,488	53.42%	+62 人
去掉 18-24 年龄段限定	+\$4,408	7.07%	+7 人

每条都带 quality_note 说明这批新增达人的质量口径（门禁判定是什么、放宽的是定向还是采购动作、风险敞口多少）——放宽不等于放水，新进清单的人同样逐条过了四层门禁。三条建议全部走包内既有函数，服务层不新增任何判定或排序逻辑。

必须跟着说清楚的一句：这条 brief 花不出去钱，是门禁只放过 7 个人造成的，不是别的原因。它命中定向的候选是 182 人，全部进了门禁与预算计算（响应里 scope.computed_on = 182、truncated_for_compute = false，见 § 6）。三条预置 brief 的预算利用率都 ≥ 99.8%——所以这条自由文本 brief 的 1.6% 是"严门禁 + 窄定向"叠出来的极端样本，我拿它当例子是因为它最能体现"钱花不出去时产品该做什么"，不是因为产品常态如此。

3. 实测：决策质量

评测口径：5,000 个合成达人，ground truth 三档（pass 3058 / review 1034 / reject 908）。

反自证设计（这块是我花时间最多的地方）：ground truth 只允许评测代码读，门禁/预算/LLM 正式链路在物理上读不到。仓库里有 AST 静态扫描 + 泄漏哨兵测试守着这条线——因为我在这个项目上被"自证"坑过两次（§ 7）。

指标	结果
水号识别（严口径）	P=0.7714 / R=0.6750 / F1=0.7200 / AUC=0.8809
三分类	准确率 0.6590 / macro F1 0.6731
pass	P=0.9466 / R=0.5386
review	P=0.4052 / R=0.9613
reject	P=0.8104 / R=0.7203

pass 的精确率 0.9466、review 的召回 0.9613 是刻意的：这套门禁的产品定位是"宁可多送一批人去人工复核，也不要让水号直接过审花钱"。所以我接受 review 精确率只有 0.4052（会多送人工），换 pass 干净和 review 不漏。这不是指标没调好，是判据取向。

分欺诈类型（能力边界要说清楚）：买粉严召回 0.797 / AUC 0.919、互动农场 0.670 / 0.880、机器评论 0.640 / 0.893，而刷播放只有 0.473 / 0.794——播放量注入在我这套特征上最难抓，如实报出。

4. 实测：业务价值——卖的不是"多赚"，是"少烧"

4.1 招牌数字，和它诚实的版本

先说裁判是谁。浪费金额与有效曝光一律按 ground truth 结算，不用引擎自己的分数（eval/audit.py 里写死，产物 method.judge 字段明写"防自证"）。否则就是"我说他是水号，所以我省了钱"的自我表扬。主口径下水号曝光按 0 计，同时给一份按 50% 计的宽松对照——两个口径同向，才说明结论不吃这条假设。

单次实验（定稿种子）：三个 campaign 合计 \$245,000 预算，少浪费 \$79,719（32.5%），有效曝光 +98.64%（6,782,758 → 13,473,202），宽松口径 +64.71%。

这个数字很好看，但我对它做了显著性检验，结果是：它是 12 个种子里第 2 高的观测，落在均值 95% CI 之外。

	单种子	12 种子
少浪费占预算	-32.5%	-21.5% ± 13.7%（CI 12.8%~30.2%，1/12 跑输基线）
有效曝光提升（主口径）	+98.6%	+97.3% ± 54.6%
有效曝光提升（宽松口径，水号曝光按 50% 计）	+64.7%	+69.1% ± 44.3%

追加 24 种子复核进一步降到 $19.1\% \pm 18.6\%$ (3/24 跑输；make multiseed SEEDS=24 可复跑，纯确定性——注意它会覆盖 output/multiseed.json 里那份 12 种子产物，跑完用 make multiseed 还原)。所以我在所有对外材料里用的是 $21.5\% \pm 13.7\%$ ，而不是继续挂 32.5%。

4.2 真正的价值不在均值，在方差

同一实验量化了另一件事：[按粉丝量选人是一场高方差赌博](#)。（口径：12 种子 $\times$ 3 campaign = 36 个观测）

	基线	KOXPilot
浪费率跨观测标准差	33.6%	4.7% (小 7.2 倍)
浪费率 > 50% 的观测	9/36 (25%)，最惨一局 99.9%	0/36，从未超过 14.8%

决定性证据在[条件胜率](#)上：把 36 个观测按"基线运气"分三档看——

基线浪费率	观测数	KOXPilot 胜率	平均少浪费
> 50% (基线踩大坑)	9	100% (9/9)	+\$57,480
5%~50% (中等)	12	91.7%	+\$13,296
< 5% (基线运气好)	15	仅 40%	-\$2,993

KOXPilot 输的 10 个观测里，基线浪费率最高只有 5.6%——也就是它只在"本来就没坑可踩"的局里输，而且输得很小；基线一旦踩坑，它 9/9 全胜。

所以这个产品的价值翻译成业务语言不是"帮你多赚 21.5%"，而是：[把"四分之一概率烧掉一半预算"这个尾部风险，换成稳定的约 4% 浪费](#)，代价是在本来就抽到好牌的局里小输一点。对低毛利的投放代理业务，消灭尾部灾难比提高均值更值钱。

两处必须自己先说的统计瑕疵，是叠加的、不是二选一：

>

① [有界比例](#)。浪费率是 [0,1] 有界比例、KOXPilot 一侧还紧贴下界，尾部不正态会让 F 检验 p 值偏乐观。

>

② [伪重复 \(pseudo-replication\)](#)。上面那 36 个观测是 12 种子 $\times$ 3 campaign，不是 36 次独立试验——同一种子下的三个 campaign 共享同一个达人库。伪重复会高估自由度、把 p 值进一步压小。所以合并后那个 $p=2.7e-21$ 我判为不可用，改报[独立单位 \(种子\) 上的检验](#)：三个 campaign 各自 n=12，方差比 p 分别为 $1.5e-07 / 1.3e-08 / 1.2e-06$ (F 值 51.6 / 81.8 / 34.9)。

>

结论因此以"标准差量级差异 + 极端频次计数 (9/36 vs 0/36) + 条件胜率"为主，这些都不依赖分布假设。两条限定与判据都写在 output/multiseed.json $\rightarrow$ B_variance_attribution.independence_unit 里，pooled_p_value_usable 字段直接标着 false，不是我临时补的说法。

4.3 这 21.5% 里，门禁到底占多少——一个推翻我自己判断的实验

原来的报告只有两条臂（基线 vs KOXPilot），于是"少浪费"这件事里混着两个原因：[不把钱压在少数头部大号上](#)（结构分散化）和[识别水号并按质量排序](#)（门禁）。谁贡献多少？答不上来。

所以我加了[第三臂](#) diversified_no_gate：同一候选池、强制 KOXPilot 的分层/地域/单人配额，但排序只看"每美元名义曝光"，[全程不读门禁判定与真实性折扣](#)。于是 总差额 = （第三臂 - 基线）+ （KOXPilot - 第三臂）。

12 种子 少浪费金额	mean ± std	为负的种子
合计	\$52.7k ± \$33.4k	1/12
其中 门禁与质量排序	\$40.0k ± \$12.6k (CV 0.31)	0/12
其中结构分散化	\$12.6k ± \$39.0k (CI 跨 0)	4/12

[稳定的那一段是门禁，不是分散投放](#)。而定稿单种子上看恰好相反（分散化 \$56.2k vs 门禁 \$23.5k）——[我原本的猜测被自己的数据推翻了](#)，这个结论我保留在仓库里而不是删掉重跑。

三条必须一起说的反面事实：

1. 第三臂的[绝对有效曝光](#)在 [12/12 个种子](#)上都高于 KOXPilot（它专挑 CPM 最便宜的长尾，KOXPilot 会主动放弃便宜但不对味的曝光）。[而且把花费归一化之后它照样赢](#)——这件事我单独写成 § 4.4，不在这里用一个分句糊过去。
2. 第三臂平均选 [516 人](#)（基线 18 / KOXPilot 181），真实采购里不可执行，它只是对照臂、不是方案。
3. "门禁贡献"其实是"门禁过滤 + 质量排序"的合计，两者共用同一批分数，[无法再拆细](#)——所以字段就叫 by_gating_and_quality_ranking，不谎报成纯门禁效果。

[一处我主动改掉的不诚实口径](#)。

界面上原来把三条臂的[绝对期望值](#)并排比。但各臂花的钱可以差几十倍（KOXPilot 只买门禁过关的人，基线臂会把预算花光），[拿绝对数并排就是拿花得多的臂去衬托花得少的臂](#)，看起来赢了其实什么都没说。所以主对比改成两个真正可比的字段：[单位效率](#)（value_per_dollar = 每美元买到的有效曝光，按 gt 结算）与[浪费金额](#)（waste_usd，与 saved_usd 同源）。绝对期望值降级为次级信息，且必须同时标明各臂花费不同。

>

举证用的是 § 2.5 那条自由文本 brief（巴西+墨西哥 Instagram 彩妆）：三臂花费 [\\$1,250 / \\$80,000 / \\$80,000](#)（差 [64 倍](#)），每美元有效曝光 KOXPilot [53.81](#) / 按粉丝量 [49.97](#) / 只分散不门禁 [43.03](#)，浪费金额 [\\$0 / \\$4,327 / \\$27,412](#)。两处必须跟着说清楚，否则这组数会被读成我不敢认的东西：① 这条 brief 花不出去钱是[门禁只放过 7 个人](#)造成的（§ 2.5），不是别的原因——三条预置 brief 的预算利用率都 [≥99.8%](#)，别把这条自由文本 brief 的 1.6% 当成产品常态；② ["每美元效率"在花费量级差 64 倍时不能直接横比](#)——这条 brief 上我领先，是因为我只买了最干净的 4 个人；等到三条预置 brief 上三臂都把预算花光，只分散不门禁那条臂的每美元有效曝光就反超我 44%~47%（见 § 4.4）。所以这个口径比绝对数诚实，但它自己也有前提，我不拿它当免检结论。

4.4 第三臂在主指标上打赢了我

我自己跑出来的第三臂在主指标上打赢了我，这里如实摆出来。[每美元有效曝光](#)是我自己选定的主口径（§ 4.3 那条 blockquote 就是我把主对比改成它的记录），在这个口径上 diversified_no_gate 三条 brief 全胜，没有一条例外。数字来自 output/audit.json → counterfactual_value_audit.per_campaign，服务 /api/plan 的 allocation.arms 是同一批值：

brief	每千美元有效曝光 KOXPilot	只分散不门禁	差距	按粉丝量基线
BRIEF-001	56,749	83,664	+47.4%	11,964
BRIEF-002	63,359	93,454	+47.5%	64,848
BRIEF-003	50,769	73,125	+44.0%	24,230

[它为什么赢](#)。不是它作弊，也不是指标不公平——这个指标是我自己定的。两层原因：一是它把同一笔钱极度摊薄到微达人身上（三条 brief 分别选 [228 / 122 / 166](#) 人，我是 69 / 39 / 73），微达人的 CPM 便宜得多；二是[门禁会挡掉一部分"便宜且名义曝光高"的号](#)——数据不完整的、真实性存疑的，第三臂全程不读门禁判定，这些库存它照买，我不买。于是我为更干净的库存付了更高的单价，价差就是这么来的。

[真正的取舍](#)。拿 BRIEF-001 并排看这笔单价买到了什么：

	KOXPilot	只分散不门禁
浪费金额 / 占预算	\$2,175 / 2.72%	\$14,599 / 18.25%
名单里的水号个数	2	41
品牌安全高风险号	0	4
花在门禁会直接 reject 的人身上	\$0	\$12,390
要签、要沟通、要交付的合约数	69 份	228 份

三条 brief 上第三臂分别买了 [4 / 2 / 4](#) 个品牌安全高风险号（我是 [0 / 0 / 0](#)），分别把 [\\$12,390 / \\$5,180 / \\$14,564](#) 花在门禁会直接判 reject 的达人身上。所以结论只能这么写：[门禁买的是确定性和品牌安全，不是每美元曝光效率的上限](#)。

[谁因此变好了](#)。这一层是我先前版本最缺的——之前只顾着自证技术指标：

1. [品牌方](#)：不用在投放结束后解释"为什么我们的品牌出现在争议内容旁边"（0 个高风险号 vs 4 个），也不用为 41 个水号的曝光付款——18.25% 的预算打水漂，结算时是要有人签字认下来的。
2. [投放执行的同学](#)：69 份合约和 228 份合约是两种工作量。228 个微达人的报价、寄样、脚本审核、发布确认、结算，在真实代理公司里基本排不进档期。第三臂的效率是在"合约与沟通成本为零"的假设下算出来的，而这个成本[不在我的指标里](#)——所以我也不能拿它去反驳第三臂，只能标明这是我的指标没度量的东西。
3. [边界坦白](#)：如果一个投放方的 KPI 真的只有每美元曝光，且品牌安全与水号曝光都不重要，[那我这套东西不该被采用](#)，应该直接用第三臂那种最大化分散度的策略。我做的是把不确定性和品牌风险压下去，这件事只有在"翻车有代价"的场景里才值钱。

BRIEF-002 上还有一处对我更不利的细节，我不省略。那条 brief 上我的浪费金额 [\\$5,494 \(12.21%\)](#) 比第三臂的 [\\$3,695 \(8.21%\)](#) 还高，有效曝光率也更低 ([87.11% vs 91.55%](#))。这说明我的门禁在那条 brief 的定向下[没有起到应有的作用](#)——3

个水号进了最终名单，是个真实的待改进项，不是口径问题。而且同一条 brief 上按粉丝量基线的每千美元有效曝光是 64,848，也高于我的 63,359，浪费金额只有 \$376 (0.84%) ——所以"我至少稳定赢按粉丝量基线"这句话在 BRIEF-002 上同样不成立，我不写那句。这条 brief 上我唯一站得住的事实是买入库存的干净程度：基线把 \$44,624 花在门禁判"需人核"的人身上，我是 \$0。

一个还没做的下一步。这两条臂的策略并不互斥：先用门禁过滤掉 review/reject，再在干净池子里最大化分散度（而不是像现在这样按质量加权价值排序），理论上可能同时拿到干净和效率。我没有实现它，也没有它的实验数据——写在这里是标明方向，不是成果。

5. 该不该用大模型：用实验回答，不用立场回答

我不认为"用了大模型"本身是能力证明。所以我做了一组控制变量实验：同一批 600 样本、同一 ground truth、同一 batch 切分，唯一变量是 system prompt。

任务是最需要语义理解的那一层：标签错配识别（他声称做 3C，实际在做什么）。

Arm	Precision	Recall	F1	token
零成本规则对照组	0.4190	0.8571	0.5625	0
Prompt v1（角色+任务描述）	0.2367	0.8453	0.3627	434,018
Prompt v2（加约束与few-shot）	0.3554	0.7857	0.4889	467,327
Prompt v3（业务判据化）	0.6136	0.7976	0.6905	492,460
最优组合 v3 × GPT-5.5	0.6875	0.7857	0.7333	—

结论是反直觉的、也是我认为最有价值的一条：v1 和 v2 都输给零成本规则。花了 90 万 token，做得比一个 Jaccard 规则还差。v3 反超的原因不是"prompt 写长了"，而是把提示词从"你是一个专业的内容审核专家，请判断……"改成业务判据：明确写清"什么叫错配"（主品类漂移 vs 兼做副品类）、给出可判定的边界、要求先输出证据再输出结论。

这件事对产品的意义：LLM 在这条链路上的价值不取决于模型多强，取决于你能不能把业务判据写清楚。判据写不清，模型不如规则；判据写清了，模型比规则高 13pp。

5.1 成本账：混合架构不是省事，是算过的

方案	LLM 调用数	代价
全 LLM（每层每人都问模型）	888	贵、慢、不稳定
KOXPilot 混合（G0/G1 纯规则，仅 brief 解析与语义适配用 LLM）	225	调用量降 3.95×
全规则（不用 LLM）	0	标签错配 F1 从 0.6824 掉到 0.5625 (-12pp)

真实 token 账（来自各 API 返回的 usage 字段，非估算）：[172 次调用](#)、[110 万 token](#)（prompt 53.7 万 / completion 56.7 万，其中 reasoning 40.3 万）。标签错配判定与 A4 适配分是[构建期真调模型](#)、[结果固化进产物](#)，服务不重算这两处。服务唯一可能实时调模型的是 A1（brief 解析）：[探到可用凭据就真调](#)、[凭据缺失/调用失败/超时一律回落确定性规则解析](#)，响应里 parse_path 写明本次走的哪条、brief.notes 写明降级原因——我这次实测走的是规则解析，原因不是没配凭据，而是模型调用被网关以 HTTP 403 model is not allowed 拒绝（§ 0 第 6 条）。有 1 个 batch 调用失败，[按"未覆盖"处理而不是填默认值](#)——填默认值等于伪造模型输出。

6. 工程完备性

测试	768 条 pytest 全绿，含反自证元测试（AST 扫描确认生产代码读不到 gt）、HTTP 接口契约测试与文档数字防漂移测试
确定性	固定种子，make eval 两次运行 metrics.json 逐字节相同 （产物刻意不含时间戳，任何指标变化都可归因到代码或数据）
双实现一致性（构建期）	判定级 5,000/5,000、证据链级 255 条 reason、预算级 9 个臂 （3 campaign × 3 臂，含第三臂对照）、审计级 3 个 campaign， diff 全为 0
双实现一致性（运行期）	真服务 vs 浏览器 TS 引擎，4 条 brief 合计 1,431 条判定 、 差异 0 条 ；explain 证据句与 CLI 逐字一致 （5 个 id，0 差异）
假设可审计	预算模型里 decay=0.7 是 建模假设不是观测值 ，补了三档敏感性扫描
服务与离线算的是 同一道题 （可自证）	契约升到 v1.1 （engine_version 1.1.0），/api/plan 响应新增只读 scope 块：recall_total / computed_on（真正进门禁与预算计算的人数， 必须等于 recall_total ） / detail_rows（返回多少条明细） / truncated_for_compute（恒为 false）。BRIEF-001 实测 746 / 746 / 60 / false —— 裁掉的只是给前端看的 60 条明细 ， 746 个候选一条不少地参与了计算 。这个字段是 § 7.2 那条 bug 的直接产物

关于 decay 扫描，结论我坚持两半一起说：[价值结论不依赖这个假设](#)（合计少浪费相对离差仅 1.4%、两段归因符号逐档不翻转），[但具体选谁会变](#)（选中名单金额加权重叠最低 0.651，低于我自设的 0.8 判据）。产物里专门有字段和测试堵住"我们验证了 0.7 是对的"这种过度解读——[这条扫描回答不了哪一档更接近真实世界](#)，那需要真实投放的重复触达数据。

但判据不达标之后，光承认"名单不唯一"是不够的——投手拿到的仍然是一份不知道哪里靠不住的清单，不不确定性被承认了，成本却全转嫁给使用者。所以我给它接了一条降级路径：[按"在几档 decay 上都被选中"](#)把名单切成两层。

交付层	人数	参照档金额	占比	动作
核心层 （三档全选中）	162	\$220,855	90.2%	可直接下单，入选不依赖 decay 取值

交付层	人数	参照档金额	占比	动作
假设敏感层（仅部分档位选中）	19	\$23,962	9.8%	标记"重复触达折扣假设敏感"，转人工确认或改按单条采购，不自动执行

结论因此从"名单不唯一"变成一句可执行的话：[这个假设的不确定性只影响 9.8% 的预算，且已经定位到具体 19 个人](#)。产物见 `output/metrics.json` → `budget_decay_sensitivity.delivery_policy`（含逐 campaign 的人员与金额区间），两层必须无损划分正式名单、两层金额之和必须等于实付金额，由 `tests/test_decay_delivery_tiers.py` 钉住。

7. 我自己抓到并修掉的问题（14 条里挑 9 条）

一个只展示漂亮指标的作品是可疑的。仓库 README 里完整列了 14 条，这里挑最能说明问题的：

问题	表现	修完的代价
评测自证 ① ：标签错配的 ground truth 实际等价于"两个集合是否相交"	规则基线 $P=R=F1=1.000$	引入隐藏真实品类后，规则基线 F1 掉到 0.562
评测自证 ② ：观测模型整块替换，正例 Jaccard 只有 0 和 1	5 个 arm 召回 精确相同 (0.857)，尽管彼此预测差异 146/600	加逐品类观测噪声后，召回重新成为真实 tradeoff
消融表里有一句能被当场证伪的话	contributes 判定只看 <code>abs(delta)</code> 不看符号，于是 G1.6（关掉它 F1 反而涨 0.0149）被算成"有贡献"，产物还写着"所有 G1 规则都有边际贡献"	改三态判定（positive/negative/negligible）， 保留 G1.6 并如实标为负贡献
一个会爆炸的比率	有效曝光提升的分母是基线有效曝光，基线烧光时分母趋 0，实测出现 +3585× ，把跨种子均值抬到 +29,977%（中位数仅 +124.5%）	改用有界口径（有效曝光率百分点差），保留旧字段并标注不稳定原因
"我们用 LLM 做语义适配"这句话在正式链路里站不住	<code>llm_cache.json</code> 里有 780 条真实调用得到的 LLM 适配分，而 <code>make eval</code> 输出的每个数字都来自规则版	补量化审计：这 780 条打分落到三个 campaign 的去重候选池（222 人）里只覆盖 33 人（14.9%） 、与既有筛选 双算率 97.1% 、覆盖子集上 LLM 分只降不升 → 结论"继续用规则版" 由判据算出 （覆盖率 $\geq 90\%$ 等三项全过会自动翻成可升格），并真跑了注入反事实（判定只翻 1 条、少浪费 \$735，不作为招牌数字）

这五条的共同点：[它们都不会报错，只会给我一个漂亮得不真实的结论](#)。我从中总结出的检查动作是——[指标好得不合理时，第一反应应该是怀疑自证，而不是庆祝](#)；多个方法结果完全相同时，先检查可分性是否已经退化。

7.1 收尾时又抓到另一类问题：结论正确，但没有产生动作

上面五条都是"数字不对"。收口阶段我发现还有一类更隐蔽的——[数字是对的，结论也是对的，但它到此为止了](#)。

第一个：敏感性扫描判出 `selection_stable = false`，我原来的处理是在文档里补一句"名单会随假设变动"。这句话没错，但它对使用者没有任何用——投手拿到的仍然是一份不知道哪里靠不住的清单，不确定性被我承认了，成本却全转嫁给了他。修法就是上面 § 6

那张分层交付表：把不确定性从"整份名单打个折扣收敛成"具体 19 个人、9.8% 的预算"。[同一份数据，一个是免责声明，一个是分流规则](#)。

第二个：`decay_scan.py` 里我特意把记录分成 blockers（会推翻结论的硬伤）和 caveats（不推翻结论但必须一起说的限定），源码注释还专门解释了"混成一个列表就会出现'扫过了没问题'这种听起来通过、实际掩盖限定条件的说法"——[结果返回值里根本没有 caveats 这个键，算完就被丢掉了](#)。本次实测 blockers 为空而 caveats 有 1 条（金额加权重叠 0.651

不达标），也就是说读者只会看到一片绿。讽刺的是这跟这个模块最初要修的那个 bug

是同一个毛病：常量注释替一段产物里不存在的输出背书。[修法](#)：把 caveats 接回产物并与 blockers 分列两个键（`metrics.json` → `budget_decay_sensitivity.caveats`），并由测试断言"`selection_stable=false` 时 caveats 不得为空、两类记录不许混装"。

第三个：README 里我写过一句"没有一个数字是手写进文档的"。这句话经不起追问——数字当然是手打进 markdown 的，只是[来源于产物](#)；真正的风险恰恰是产物重跑后数值变了，而文档里的字不会自己改。我没有把这句话改软，而是补了 `tests/test_doc_numbers.py`：把文档里的招牌数字逐条钉到产物字段上，连千分位、百分号、有效位都按人眼看到的字符串断言，还包括一条"文档声称的测试总数必须等于实际收集到的数量"。[不改口气，改机制](#)。

这三条我认为比前五条更值得说。前五条是技术功底问题，这三条是产品意识问题：[一个结论如果不能推导出下一步动作，那它在产品里就还没做完](#)。

7.2 最后抓到的一条：展示参数悄悄改变了计算语义

这条是收口最后阶段抓到的，性质又不一样——[代码不报错，响应结构完整，数字看起来还挺合理](#)，但服务和离线在算两道不同的题。

- **症状：**同一条 BRIEF-001，离线 CLI 选 69 人花 \$79,855（利用率 99.82%），线上服务只花 \$5,693（7.12%）——[预算利用率差 14 倍](#)。
- **根因：**服务的 `_recall` 把 `options.top_n`（一个[展示用](#)参数，默认 200）当成了召回上限。BRIEF-001 命中定向 746 人，被截到 200 之后，这个残缺的池子直接进了门禁和分配器；而分配器的结构配额（长尾 $\geq 25\%$ / 单国 $\leq 60\%$ / 单人 $\leq 20\%$ ）是在[候选池上求解](#)的，小池里"买得起又能满足配额"的库存跟着消失，于是[无可行解](#)，钱花不出去。
- **修法：**截断只作用于[展示](#)——`top_n / explain_limit` 只裁返回的候选明细条数，召回一条不裁。服务升 1.1.0、契约升 v1.1，并新增只读 `scope` 块把"参与计算的候选没被裁"写进响应体（§ 6）。[src/koxpilot/一行未改](#)：错的一直只是服务层那一次切片，768 条测试也因此全程是绿的——这恰恰是这类 bug 的可怕之处。
- **它给我的教训：**["展示参数悄悄改变了计算语义"这类 bug](#)，只有把两条路径的中间量逐个对齐才能发现，看结果对不对是发现不了的。
服务返回的清单、金额、约束检查全都自治，`all_enforced_satisfied` 甚至是 `true`；只有把服务和 CLI

的[候选池条数](#)摆在一起，才看出一边 746、一边 200。所以我没有只改那行切片，而是把中间量（recall_total / computed_on）提成响应里的一等字段——[下次再出这种事，它得自己露出来](#)。

8. 已知弱项与边界（不藏在附录）

弱项	数字	我的判断	
数据是合成的	—	这是最大边界。所有绝对值只在我的数据生成假设内成立。唯一能推翻或确认结论的实验是接一批真实达人 + 人工标注重跑	
头部大号识别差	follower_bucket=mega F1 0.467	系统性短板 （11/12 种子进榜，p=1.5e-07），不是噪声	
单次弱项清单不可信	country=SA、`youtube\`	macro`	是 采样噪声 （p=0.132 / 0.803）。典型 winner's curse：单次实验里最差的那几格，多数只是运气差
阈值敏感性不稳健	±20% 最大 F1 偏移 0.0582，stable=False	如实上报，没有调宽判据让它变 True	
刷播放最难抓	严召回 0.473	播放注入在现有特征上区分度不足	
A4 LLM 未接正式链路	覆盖率 14.9%	明确改口径为"规则版在生产链路、LLM 版构建期跑通并固化"，而不是让文档继续说 LLM	
A1 走规则解析	调用被拒 HTTP 403	llm_runtime.available 是 true（凭据在），但该模型不被允许调用，A1 因此落回确定性规则解析。响应里 parse_path=rule、brief.notes 逐字写明 403 原文，不藏。换一份有权限的凭据就会走 LLM 路径——是环境权限问题，不是功能缺失	
BRIEF-002 上门禁没起到应有作用	3 个水号进名单，浪费 12.21%	真实待改进项：该 brief 上我的浪费率高于只分散不门禁臂（8.21%）、也高于按粉丝量基线（0.84%），有效曝光率 87.11% 三臂最低。是 G1 在那条定向下没抓住，不是口径问题（§ 4.4）	
服务是演示实例	—	单实例、无副本、无灰度、只读不写； 冷启动首个请求要等数据集懒加载 。它证明的是"这条链路能被 HTTP 真调"，不是"这套东西扛得住生产流量"	

弱项	数字	我的判断	
名单不唯一	金额加权重叠最低 0.651	换 decay 假设就换一批人。已接降级路径：按假设敏感性分两层交付，不确定性收敛到 9.8% 的预算、19 个人（见 § 6）	
预算分配非最优	—	是"边际性价比贪心 + 结构约束修正"，叠加 5 条约束后不保证最优，我没做 MILP 对照，所以说不出离最优差多少	

9. 我想说明的三件事

1. 我做产品的方式是"先能被追问，再能被展示"。这个作品里我花在评测、反自证、显著性检验上的时间，超过花在功能上的时间。因为一个投放决策产品，最终要被投放执行的人问"你凭什么让我砍掉这个达人"，被老板问"你说少烧的钱，怎么算的"。答不上来的产品不会被用。
2. 我把 AI 当成一个需要被度量的组件，不是一个卖点。这条链路上我真正拿 LLM 做的只有两件事（brief 结构化、语义适配），其余全是规则——因为我算过：全 LLM 是 3.95 倍调用量，而 G0/G1 用规则一分不差。同时我也用实验证明了：判据写不清的时候，LLM 是不如规则的。
3. 我愿意把不利的结论写进交付物。这份文档里有三处我自己的判断被自己的实验推翻（价值幅度被单次高估、价值来源不是分散化而是门禁、单次弱项清单大半是噪声），有一条负贡献规则我留在系统里如实标注。更彻底的一条是 § 4.4：我自己加的对照臂在我自己定的主指标上赢了我，我把它连同"谁因此变好了"和"什么情况下不该用我这套东西"一起写在正文里，而不是删掉那条臂。我认为这比多报几个百分点有用。

附录：复现方式

```
git clone <repo> && cd koxpilot
pip install -e .

make all # 数据 → 门禁 → 预算 → 评测（固定种子，无网络依赖，约 1 分钟）
make test # 768 条测试
make multiseed # 12 种子稳健性实验（纯确定性，约 30s）

# 本地起 HTTP 服务（就是线上那个）
make api-bundle # 把 src/koxpilot + data/ + thresholds 打进 api/_bundle/
make api-serve # uvicorn 起在 http://127.0.0.1:8399，公开、不需要 token
make api-parity # ① 服务 vs 浏览器 TS 引擎逐条比对 ② 服务 vs CLI explain 逐字比对

# 需要 API key（构建期实验，可选）
make llm # 真实 LLM runner
```

```
make promptbench # Prompt v1/v2/v3 × 双模型横评
```

起 HTTP 服务后可以直接 [curl](#)（无鉴权，不需要任何 key）：

```
pip install -r api/requirements.txt
cd api && python3 main.py # → http://127.0.0.1:8000
curl -s http://127.0.0.1:8000/api/health
curl -s http://127.0.0.1:8000/api/kox/KOX-000002/explain
curl -s -X POST http://127.0.0.1:8000/api/plan \
  -H 'Content-Type: application/json' \
  -d '{"brief_text": "巴西和墨西哥的 Instagram 彩妆, 18-24 女性, 预算 8 万美金, 避开 Focallure"}'
```

注意 `api/run.sh` **不是**本地启动脚本：它是部署平台自动生成的，里面会 `cp -r /opt/bytfaas/*`，只在部署环境里成立，在自己电脑上跑必然失败。本地用上面这两条，或者用 `make api-serve`（`uvicorn` 起在 8399）。另外把 `api/_bundle/` 整个移走也能跑通——路径解析是「`api/_bundle` 优先、回落仓库根」，clone 下来没有部署产物的情形同样成立，实测 BRIEF-001 仍返回 69 人 / \$79,855.29。

产物对照：`output/metrics.json`（六张表 + 反事实审计 + 诚实性清单）、`output/audit.json`（gt 口径的三臂价值审计）、`output/multiseed.json`（12 种子）、`output/prompt_bench.json`（Prompt 横评）、`output/llm_bench.json`（token 实账）、`web/public/data/consistency.json`（双实现全库比对）。

[文档索引](#)：docs/01 架构 | docs/02 四层门禁 | docs/03 评测方法 | docs/04 Prompt 与成本 | docs/05 边界与已知问题 | docs/06 稳健性。